

Dart Programming

Object Oriented Programming

Class – Object

- Dart is an object-oriented programming language, so it supports the concept of class, object ... etc. In Dart, we can define classes and objects of our own.
- We use the class keyword to do so. Dart supports object-oriented programming features like classes and interfaces.
- An object is an abstraction meant to represent a component of a program.
- Objects can be created, destroyed, given attributes, and made to perform actions.
- The metaphors that they can represent are really limited only by your imagination.
- Objects have instance variables, which represent their attributes, and methods, which represent the actions they can perform.
- Every object belongs to a class. The class of an object is its type. In other words, objects of the same class are the same type.
- Objects of the same class have the same instance variables and methods available to them (although the value of those instance variables may differ).

References to Objects and Instances

- Variables are references to objects.
- The new operator is used for instantiating new objects. An *instance* of a class is a unique object created of that class type.
- When an object is created (using the new operator, or literal syntax), a region of memory is allocated large enough to fit all of the instance variables and other data associated with that object.
- The variable that refers to the object in itself does not use very much memory.
- It's just a pointer to the address in memory where the object resides. It is, therefore, easy for multiple variables to refer to the same object.

Defining class

- Class is the blueprint of objects and class is the collection of data members and data function means which include these fields, getter and setter, and constructor and functions.
- Declaring class in Dart

```
class class_name {  
    // Body of class  
}
```

- In the above syntax:
- Class is the keyword used to initialize the class.
- class_name is the name of the class.
- The body of the class consists of fields, constructors, getter and setter methods, etc.

- Classes Fields are the variables which data for the objects. Let us check with an Example:

```
class Student{  
  
    // Fields defining the  
    // Properties of Class  
    int? roll_no;  
    String? name;  
  
}
```

Class methods

- Class Methods are the functions that provide behavior for an object. Let us check with a Example:

```
class Student{  
    // Fields defining the  
    // Properties of Class  
    int? roll_no;  
    String? name;  
  
    void print_name(){  
        print("Student Name: $name");  
    }  
}
```

- What's special about them, as opposed to regular functions, is that they have access to the instance variables of the class they are a part of.

- Objects are the instance of the class and they are declared by calling the constructor of the class
- `var object_name = class_name([ arguments ] );`
- In the above syntax:
 - `object_name` is the name of the object and its naming is similar to the variable name in dart.
 - `class_name` is the name of the class whose instance variable is been created.
 - `arguments` are the input which are needed to be pass if we are willing to call a constructor
- After the object is created, there will be the need to access the fields which we will create. We use the `dot(.)` operator for that purpose.

```
// For accessing the property  
object_name.property_name;  
  
// For accessing the method  
object_name.method_name();
```

Constructors

- Dart also provides the support of constructors. Constructors are a special method that is used to initialize an object when created in the program.
- In object-oriented programming when an object is created, it automatically calls the constructor. All classes have their default constructor which is created by the compiler when class is called, moreover one can also define constructor of its own. But, you must note that if you do so then the default constructor will not be created and will be ignored.
- Constructors in Dart: The constructors have the same name as the class name and don't have any return type.
- A Constructor is a block of code that initializes the state and values during object creation. Constructor is name same as the class name and doesn't return any value.

```
class_name( [ parameters ] ){  
    // Constructor Body  
}
```

Constructor Example

- A basic constructor is a method with the same name as the class. It can take parameters and is most often used for assigning an object's instance variables.
- `/// ns` is for number of sides, `nd` is for number of dice
- `Dice(int ns, int nd) {`
 - `sides = ns;`
 - `numberOfDice = nd;`
 - `}`
- And here's how this new constructor can be called at instantiation time.
- `Dice d = new Dice(6, 4);` // d has 6 sides and 4 dies

- The keyword `this` is used for referring to the instance of a class currently operated on.
- By using `this`, we can distinguish between instance variables and local variables. If there is a local variable named `numberOfDice`, then within the block where it is declared, `numberOfDice` refers to that local variable, while `this.numberOfDice` refers to the instance variable.
- We can write a verbose constructor using the same names as our variables, as follows:
 - `Dice(int sides, int numberOfDice) {`
 - `_sides = sides;`
 - `this.numberOfDice = numberOfDice;`
 - `}`

```
// Dart program to illustrate
// this keyword
void main()
{
    Student s1 = new Student('S001');
}

class Student
{
    // defining local st_id variable
    var st_id;
    Student(var st_id)
    {
        // using this keyword
        this.st_id = st_id;
        print("GFG - Dart THIS Example");
        print("The Student ID is : ${st_id}");
    }
}
```

Named Constructors

- Named Constructor: As you can't define multiple constructors with the same name, this type of constructor is the solution to the problem. They allow the user to make multiple constructors with a different name.
- If we would like more than one way to construct an object at initialization time, we need to use named constructors.
- Named constructors are regular constructors that also have an identifier associated with them.

```
class_name.constructor_name ( parameters ){  
    // Body of Constructor  
}
```

- `class Temperature {`
- `double degreesKelvin;`
- `Temperature(this.degreesKelvin);`
- `Temperature.fromCelsius(double degreesCelsius) {`
- `degreesKelvin = degreesCelsius + 273.15;`
- `}`
- `Temperature.fromFahrenheit(double degreesFahrenheit) {`
- `degreesKelvin = 5 / 9 * (degreesFahrenheit - 32) + 273.15;`
- `}`
- `}`

Libraries

- Dart has a concept of *libraries*. Libraries can span multiple files, and the code in each can access the code in the others.
- The keyword `library` is used for declaring a library. It is followed by the name of the library, any import statements that the library needs, and a listing of the files that constitute the library, each preceded by the keyword `part`.
- Those files then have a part of statement at the top of their source. All of the files in a library have access to the packages imported.
 - `library pig;`
 - `import "dart:math"; // for Random`
 - `import "dart:io"; // for stdin`
 - `part "dice.dart";`
 - `part "player.dart";`

Class variable and class method

- Every instance of a class has its own copy of that class's instance variables.
- A single copy of a class variable exists for all instances of a class.
- It is referred to directly through the class's name and the (.) operator. Class methods cannot access instance variables.
- They can only work with class variables or local variables. They are also accessed through the class name and the . operator.
- The keyword static is used for declaring a class variable or class method.
- The static keyword is used for memory management of global data members. The static keyword can be applied to the fields and methods of a class. The static variables and methods are part of the class instead of a specific instance.
- The static keyword is used for a class-level variable and method that is the same for every instance of a class, this means if a data member is static, it can be accessed without creating an object.
- The static keyword allows data members to persist Values between different instances of a class.
- There is no need to create a class object to access a static variable or call a static method: simply put the class name before the static variable or method name to use them.

Example

- `class SodaCan {`
- `static double price = 1.00;`
- `static double capacity = 350.0;`
- `bool dented = false;`
- `void drink() => print("Mmm refreshing.");`
- `static double calculateCostPerMilliliter() {`
- `return price / capacity;`
- `}`
- `}`
- `void main() {`
- `SodaCan.price = 2.00; // all soda cans cost 2.00`
- `SodaCan myCola = new SodaCan();`
- `print(SodaCan.calculateCostPerMilliliter()); // prints 2.00 / 350`
- `myCola.calculateCostPerMilliliter(); // illegal, method can't be accessed by instances`
- `myCola.price = 1.50; // illegal, price can only be accessed through the class name`
- `SodaCan.drink(); // illegal, instance method can't be used statically`
- `}`

Example

- // Dart Program to show
- // Static methods in Dart
- class Employee {
- static var emp_dept;
- var emp_name;
- int emp_salary;
-
- // Function to show details
- // of the Employee
- showDetails() {
- print("Name of the Employee is: \${emp_name}");
- print("Salary of the Employee is: \${emp_salary}");
- print("Dept. of the Employee is: \${emp_dept}");
- }
- }
-

```
// Main function
void main() {
    Employee e1 = new Employee();
    Employee e2 = new Employee();
    Employee.emp_dept = "MIS";

    print(" Dart static Keyword Example");
    e1.emp_name = 'Rahul';
    e1.emp_salary = 50000;
    e1.showDetails();

    e2.emp_name = 'Tina';
    e2.emp_salary = 55000;
    e2.showDetails();
}
```

Encapsulation

- Encapsulation is one of the important concepts of object-oriented programming.
- In Dart, Encapsulation means hiding data within a library, preventing it from outside factors.
- It helps you control your program and prevent it from becoming too complicated.
- By default, every .dart file is a library. A library is a collection of functions and classes. A library can be imported into another library using the import keyword.
- Encapsulation can be achieved by:
 - Declaring the class properties as private by using underscore(_).
 - Providing public getter and setter methods to access and update the value of private property.
- Dart doesn't support keywords like public, private, and protected. Dart uses _ (underscore) to make a property or method private. The encapsulation happens at library level, not at class level.

Why encapsulation important

- Data Hiding: Encapsulation hides the data from the outside world. It prevents the data from being accessed by the code outside the class. This is known as data hiding.
- Testability: Encapsulation allows you to test the class in isolation. It will enable you to test the class without testing the code outside the class.
- Flexibility: Encapsulation allows you to change the implementation of the class without affecting the code outside the class.
- Security: Encapsulation allows you to restrict access to the class members. It will enable you to limit access to the class members from the code outside the library.

Getters and Setters methods

- Getter and setter methods are used to access and update the value of private property.
- Getter methods are used to access the value of private property. Setter methods are used to update the value of private property.
- `return_type get property_name {`
 - `// Getter body`
 - `}`
- Note: Instead of writing `{ }` after the property name, you can also write `=>` (fat arrow) after the property name.
- We can use getters to implement some data validation in the private field

```
• class NoteBook {  
• // Private properties  
• String _name;  
• double _prize;  
• // Constructor  
• NoteBook(this._name, this._prize);  
• // Getter to access private property _name  
• String get name {  
• if (_name == "") {  
• return "No Name";  
• }  
• return this._name;  
• }  
• // Getter to access private property _prize  
• double get price {  
• return this._prize;  
• }  
• }
```

```
void main() {  
    // Create an object of NoteBook class  
    NoteBook nb = new NoteBook("Apple", 1000);  
    print("First Notebook name: ${nb.name}");  
    print("First Notebook price: ${nb.price}");  
    NoteBook nb2 = new NoteBook("", 500);  
    print("Second Notebook name: ${nb2.name}");  
    print("Second Notebook price: ${nb2.price}");  
}
```

- Setter is used to set the value of a property. It is mostly used to update a private property's value.
- Setter provide explicit write access to an object properties.
- `set property_name (value) {`
 - `// Setter body`
 - `}`
- Note: Instead of writing `{ }` after the property name, you can also write `=>` (fat arrow) after the property name.
- We can use setters to implement some data validation before Changing the fields

Example

```
class NoteBook {  
    // Private properties  
    String? _name;  
    double? _prize;  
    // Setter to update the value of name property  
    set name(String name) => _name = name;  
    // Setter to update the value of price property  
    set price(double price) {  
        if (price < 0) {  
            throw Exception("Price cannot be less than 0");  
        }  
        this._prize = price;  
    }  
    // Method to display the values of the properties  
    void display() {  
        print("Name: $_name");  
        print("Price: $_prize");  
    }  
}
```

```
void main() {  
    // Create an object of NoteBook class  
    NoteBook nb = new NoteBook();  
    // setting values to the object using setter  
    nb.name = "Dell";  
    nb.price = 250;  
  
    // Display the values of the object  
    nb.display();  
}
```